

```
#This is the dataset that will be analyzed in order to investigate how loan applicatins are denied or approved
#based off of the applicants variables
```

```
#This dataset is a version of original Credit Risk dataset on Kaggle
#and had additional variables added to this version of the Credit Risk dataset based on Financial Risk for Loan Approval data
#These values are based off of categorical, ordinal, and continuous variables
```

```
import pandas as pd
import numpy as np
!pip install xgboost
df = pd.read_csv("/content/Classification.csv")
```

```
print(df.head())
print(df.info())
print(df.isnull().sum())
```

```
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.1.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.28.9)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)
```

```
   person_age person_gender person_education person_income person_emp_exp \
0         22.0         female           Master       71948.0             0
1         21.0         female    High School       12282.0             0
2         25.0         female    High School       12438.0             3
3         23.0         female    Bachelor       79753.0             0
4         24.0          male           Master       66135.0             1
```

```
   person_home_ownership loan_amnt loan_intent loan_int_rate \
0                RENT    35000.0  PERSONAL       16.02
1                OWN     1000.0  EDUCATION       11.14
2            MORTGAGE     5500.0   MEDICAL       12.87
3                RENT    35000.0   MEDICAL       15.23
4                RENT    35000.0   MEDICAL       14.27
```

```
   loan_percent_income cb_person_cred_hist_length credit_score \
0                0.49                3.0           561
1                0.08                2.0           504
2                0.44                3.0           635
3                0.44                2.0           675
4                0.53                4.0           586
```

```
   previous_loan_defaults_on_file loan_status
0                          No             1
1                          Yes             0
2                          No             1
3                          No             1
4                          No             1
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 45000 entries, 0 to 44999
```

```
Data columns (total 14 columns):
```

#	Column	Non-Null Count	Dtype
0	person_age	45000 non-null	float64
1	person_gender	45000 non-null	object
2	person_education	45000 non-null	object
3	person_income	45000 non-null	float64
4	person_emp_exp	45000 non-null	int64
5	person_home_ownership	45000 non-null	object
6	loan_amnt	45000 non-null	float64
7	loan_intent	45000 non-null	object
8	loan_int_rate	45000 non-null	float64
9	loan_percent_income	45000 non-null	float64
10	cb_person_cred_hist_length	45000 non-null	float64
11	credit_score	45000 non-null	int64
12	previous_loan_defaults_on_file	45000 non-null	object
13	loan_status	45000 non-null	int64

```
dtypes: float64(6), int64(3), object(5)
```

```
memory usage: 4.8+ MB
```

```
None
```

```
person_age      0
person_gender    0
person_education 0
person_income    0
person_emp_exp   0
```

Lo, T. (2024). Loan Approval Classification Dataset. Kaggle.com. <https://www.kaggle.com/datasets/taweilo/loan-approval-classification-data>

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix, roc_auc_score
)
import matplotlib.pyplot as plt
import seaborn as sns

#Select target variable (loan_status)
#Separate features from the target variable
#Encode categorical features using one-hot encoding
#drop_first avoids multicollinearity
#Split data into training and testing sets
#80% training and 20% testing
#Stratify maintains class balance
#Display training data shape to verify split

y = df["loan_status"]
X = df.drop("loan_status", axis=1)

X_encoded = pd.get_dummies(X, columns=X.select_dtypes(include=['object']).columns, drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2, random_state=42, stratify=y
)

print("Shape:", X_train.shape)

Shape: (36000, 22)

```

scikit-learn. (2018). sklearn.model_selection.train_test_split — scikit-learn 0.20.3 documentation. Scikit-Learn.org. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

```

#Logistic Regression

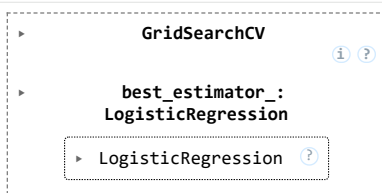
#GridSearchCV tests different parameter combinations
#cv=5 uses 5-fold cross-validation for model evaluation
#Scoring metric: ROC-AUC

#Hyperparameters:
#C: regularization strength (smaller = stronger regularization)
#penalty: L2 regularization to prevent overfitting
#solver: optimization algorithm compatible with L2 penalty

#GridSearchCV selects the best Logistic Regression model based on ROC-AUC performance

lr_param = {
    "C": [0.1, 1, 10],
    "penalty": ["l2"],
    "solver": ["liblinear"]
}
lr = LogisticRegression(random_state=42, max_iter=1000)
lr_cv = GridSearchCV(lr, lr_param, cv=5, scoring="roc_auc")
lr_cv.fit(X_train, y_train)

```



scikit-learn. (2014). LogisticRegression. Scikit-Learn.org. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

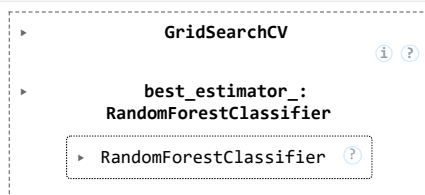
```
#Random Forest

#GridSearchCV tests different parameter combinations
#cv=5 uses 5-fold cross-validation for model evaluation
#Scoring metric: ROC-AUC

#Hyperparameters:
#n_estimators: number of decision trees in the forest (fixed at 100)
#max_depth: maximum depth of each tree (tested at 5 and 10 to control overfitting)
#class_weight: set to "balanced" to automatically adjust weights inversely proportional to class frequencies

#GridSearchCV selects the best Random Forest model based on ROC-AUC performance

rf_param = {
    "n_estimators": [100],
    "max_depth": [5, 10],
    "class_weight": ["balanced"]
}
rfc = RandomForestClassifier(random_state=42)
rfc_cv = GridSearchCV(rfc, rf_param, cv=5, scoring="roc_auc")
rfc_cv.fit(X_train, y_train)
```



scikit-learn. (2024). RandomForestClassifier. Scikit-Learn.org. <https://scikit-learn.org/1.6/modules/generated/sklearn.ensemble.RandomForestClassifier.html>.

```
#XGBoost Classifier

#Gradient boosting model that builds trees sequentially
#Each tree focuses on correcting previous prediction errors
#GridSearchCV used to tune model hyperparameters
#cv=5 applies 5-fold cross-validation
#Scoring metric: ROC-AUC

#Hyperparameters:
#max_depth: controls tree complexity
#learning_rate: controls how much each tree contributes
#n_estimators: number of boosting trees
#gamma: minimum loss reduction needed to split a node
#

#GridSearchCV selects the best Random XGBoost based on ROC-AUC performance

xgb_param = {
    "max_depth": [3, 5],
    "learning_rate": [0.1],
    "n_estimators": [100],
    "gamma": [0]
}
xgb = XGBClassifier(random_state=42, eval_metric="logloss", use_label_encoder=False)
xgb_cv = GridSearchCV(xgb, xgb_param, cv=5, scoring="roc_auc")
xgb_cv.fit(X_train, y_train)
```

```
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:199: UserWarning: [13:38:21] WARNING: /workspace/src/learner.cc:790:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:199: UserWarning: [13:38:21] WARNING: /workspace/src/learner.cc:790:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:199: UserWarning: [13:38:21] WARNING: /workspace/src/learner.cc:790:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:199: UserWarning: [13:38:21] WARNING: /workspace/src/learner.cc:790:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:199: UserWarning: [13:38:22] WARNING: /workspace/src/learner.cc:790:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:199: UserWarning: [13:38:22] WARNING: /workspace/src/learner.cc:790:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:199: UserWarning: [13:38:22] WARNING: /workspace/src/learner.cc:790:
Parameters: { "use_label_encoder" } are not used.
```

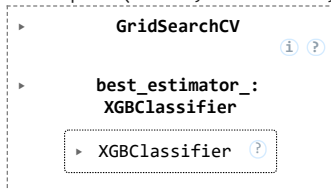
```
bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:199: UserWarning: [13:38:23] WARNING: /workspace/src/learner.cc:790:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:199: UserWarning: [13:38:23] WARNING: /workspace/src/learner.cc:790:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:199: UserWarning: [13:38:24] WARNING: /workspace/src/learner.cc:790:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:199: UserWarning: [13:38:24] WARNING: /workspace/src/learner.cc:790:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
```



Python API Reference — xgboost 1.7.5 documentation. (n.d.). Xgboost.readthedocs.io.

https://xgboost.readthedocs.io/en/stable/python/python_api.html#xgboost.XGBClassifier

scikit-learn. (2019). sklearn.metrics.confusion_matrix — scikit-learn 0.21.3 documentation. Scikit-Learn.org. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html

```

#Model evaluation and comparison
#Evaluate_classification evaluates trained classification models on test data

#Generates predictions from the model
#Prints performance results for easy interpretation
#Visualizes results using a confusion matrix

#This evaluation helps determine which model performs best on unseen data and supports model selection
#Trained classification models (Logistic Resregion, Random Forest, XGBoost)
#X_test: features for testing
#y_test: true target values for testing

```

```
def evaluate_classification(name, model, X_test, y_test):
    pred = model.predict(X_test)
    pred_proba = model.predict_proba(X_test)[:, 1]
    auc = roc_auc_score(y_test, pred_proba)
    acc = accuracy_score(y_test, pred)
    print(f"\n{name}")
    print(f"→ AUC: {auc:.3f}, Accuracy: {acc:.3f}")
    print(classification_report(y_test, pred))

    cm = confusion_matrix(y_test, pred)
    plt.figure(figsize=(5,4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.title(f"{name} - Confusion Matrix")
    plt.ylabel("True")
    plt.xlabel("Predicted")
    plt.show()
    plt.tight_layout()

    return {"Model": name, "AUC": auc, "Accuracy": acc}

eval_results = []
eval_results.append(evaluate_classification("Logistic Regression", lr_cv.best_estimator_, X_test, y_test))
eval_results.append(evaluate_classification("Random Forest", rfc_cv.best_estimator_, X_test, y_test))
eval_results.append(evaluate_classification("XGBoost", xgb_cv.best_estimator_, X_test, y_test))

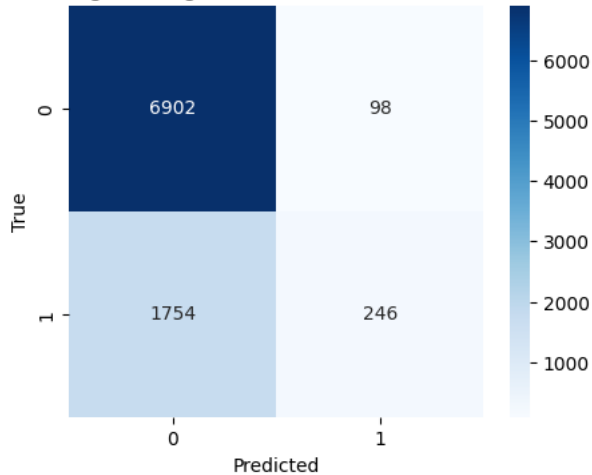
results_df = pd.DataFrame(eval_results)
print("\n Summary Table:")
print(results_df)
```

Logistic Regression

→ AUC: 0.780, Accuracy: 0.794

	precision	recall	f1-score	support
0	0.80	0.99	0.88	7000
1	0.72	0.12	0.21	2000
accuracy			0.79	9000
macro avg	0.76	0.55	0.55	9000
weighted avg	0.78	0.79	0.73	9000

Logistic Regression - Confusion Matrix



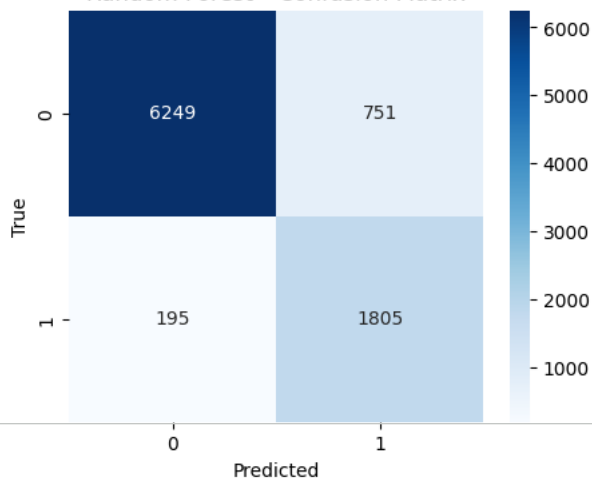
Random Forest

→ AUC: 0.971, Accuracy: 0.895

	precision	recall	f1-score	support
0	0.97	0.89	0.93	7000
1	0.71	0.90	0.79	2000
accuracy			0.89	9000
macro avg	0.84	0.90	0.86	9000
weighted avg	0.91	0.89	0.90	9000

<Figure size 640x480 with 0 Axes>

Random Forest - Confusion Matrix



XGBoost

→ AUC: 0.975, Accuracy: 0.929

	precision	recall	f1-score	support
0	0.94	0.97	0.96	7000
1	0.89	0.77	0.83	2000
accuracy			0.93	9000
macro avg	0.92	0.87	0.89	9000
weighted avg	0.93	0.93	0.93	9000